

IF25-21013 - Strategi Algoritma

Laporan Tugas Besar

*Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Robocode Tank
Royale*

Kelompok 18

BUKAN ORANG KAMPUNG



Kelas : RC

Semester : Genap 2025/2026

Dosen : Winda Yulita, M.Cs

Asisten Dosen : Devina Kartika

Disusun Oleh:

M.Rif'at Syauki	124140138
Pina Ramanda	124140170
Chesya Margaretha Deto	124140098

Program Studi Teknik Informatika

Institut Teknologi Sumatera

2026

DAFTAR ISI

BAB 1 Deskripsi Tugas	4
BAB 2 Landasan Teori	5
2.1 Dasar Teori Algoritma Greedy.....	5
2.1.1 Himpunan Kandidat	5
2.1.2 Himpunan Solusi	5
2.1.3 Fungsi Solusi	5
2.1.4 Fungsi Seleksi	5
2.1.5 Fungsi Kelayakan	6
2.1.6 Fungsi Objektif.....	6
2.2 Cara Kerja Program Secara Umum	6
2.2.1 Siklus Pertandingan	6
2.2.2 Pemrosesan Informasi Arena	7
2.2.3 Pengambilan Keputusan Greedy.....	7
2.2.4 Eksekusi Aksi Bot.....	7
2.2.5 Menjalankan Program pada Game Engine	8
BAB 3 Aplikasi Strategi Greedy.....	9
3.1 Pemetaan Persoalan ke Elemen Algoritma Greedy	9
3.1.1 Himpunan Kandidat	9
3.1.2 Himpunan Solusi	9
3.1.3 Fungsi Solusi	9
3.1.4 Fungsi Seleksi	9
3.1.5 Fungsi Kelayakan	10
3.1.6 Fungsi Objektif.....	10
3.2 Eksplorasi Empat Alternatif Solusi Greedy.....	10
3.2.1 Bot Alternatif 1: CodeCey	10
3.2.2 Bot Alternatif 2: AdaptiveBot	11
3.2.3 Bot Alternatif 3: Bot2	12
3.2.4 Bot Alternatif 4: Bot1	13
3.3 Strategi Greedy yang Dipilih.....	14
BAB 4 Implementasi dan Pengujian	15
4.1 Implementasi Empat Alternatif Solusi.....	15
4.1.1 Bot Alternatif 1: CodeCey	15
4.1.2 Bot Alternatif 2: AdaptiveBot	17
4.1.3 Bot Alternatif 3: Bot2	18
4.1.4 Bot Alternatif 4: Bot1	19
4.2 Struktur Data, Fungsi, dan Prosedur Bot	21

4.3	Pengujian Bot	22
4.4	Analisis Hasil Pengujian.....	23
	BAB 5 Kesimpulan dan Saran.....	25
5.1	Kesimpulan	25
5.2	Saran.....	25
	BAB A Lampiran.....	26
A.1	Tautan Repository GitHub.....	26

BAB 1

DESKRIPSI TUGAS

Berdasarkan dokumen spesifikasi tugas besar, ketentuan wajib pengerjaan tugas adalah sebagai berikut.

- Buatlah 4 bot (1 utama dan 3 alternatif, tetap dikumpulkan 1 bot yang terbaik saja) dalam bahasa C# (.NET) yang mengimplementasikan algoritma Greedy pada bot permainan *Robocode Tank Royale* dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota minimal 2 orang dan maksimal 3 orang.
- Strategi greedy yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan di atas.
- Strategi greedy yang diimplementasikan harus berbeda untuk setiap bot yang diimplementasikan dan setiap strategi greedy harus menggunakan *heuristic* yang berbeda.
- Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH, baik dari *starter pack* maupun dari repository *engine* asli.
- Buatlah strategi greedy terbaik, karena setiap bot utama dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi greedy yang kelompok Anda buat harus dijelaskan dan ditulis secara eksplisit pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam-macam dalam menyusun strategi greedy untuk memenangkan permainan. Implementasi pemain harus dapat dijalankan pada *game engine* yang telah disebutkan di atas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi greedy yang disebutkan, harus dilengkapi dengan kode sumber yang dibuat. Artinya, semua strategi harus diimplementasikan.
- Mahasiswa disarankan membaca dokumentasi dari *game engine*. Perlu diperhatikan bahwa *game engine* yang digunakan untuk tubes ini SUDAH DIMODIFIKASI. Perubahan dapat dilihat pada bagian *starter pack*.

BAB 2

LANDASAN TEORI

2.1 Dasar Teori Algoritma Greedy

Algoritma greedy adalah paradigma perancangan algoritma yang membangun solusi secara bertahap dengan memilih keputusan terbaik pada kondisi saat ini. Keputusan tersebut bersifat lokal, sehingga algoritma tidak mengevaluasi seluruh kemungkinan solusi dari awal hingga akhir. Pendekatan ini banyak digunakan pada persoalan optimasi karena sederhana, cepat, dan mudah diimplementasikan [1, 2].

Dalam algoritma greedy, terdapat beberapa elemen utama yang perlu didefinisikan agar proses pengambilan keputusan menjadi jelas. Elemen-elemen tersebut dijelaskan sebagai berikut.

2.1.1 Himpunan Kandidat

Himpunan kandidat adalah kumpulan seluruh pilihan yang mungkin dipilih untuk membentuk solusi. Kandidat dapat berupa objek, langkah, aksi, atau keputusan yang tersedia pada setiap tahap penyelesaian masalah.

2.1.2 Himpunan Solusi

Himpunan solusi adalah kumpulan kandidat yang telah dipilih dan membentuk solusi parsial atau solusi akhir. Himpunan ini terus dibangun secara bertahap sesuai keputusan yang diambil oleh algoritma.

2.1.3 Fungsi Solusi

Fungsi solusi digunakan untuk memeriksa apakah himpunan solusi yang terbentuk sudah memenuhi kondisi akhir atau tujuan penyelesaian masalah. Jika kondisi akhir belum terpenuhi, algoritma masih perlu memilih kandidat berikutnya.

2.1.4 Fungsi Seleksi

Fungsi seleksi menentukan kandidat terbaik yang akan dipilih pada suatu tahap. Pemilihan ini dilakukan berdasarkan kriteria atau heuristik tertentu yang dianggap paling menguntungkan pada kondisi saat itu.

2.1.5 Fungsi Kelayakan

Fungsi kelayakan memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam solusi tanpa melanggar batasan masalah. Kandidat yang tidak layak akan diabaikan dan tidak dimasukkan ke dalam himpunan solusi.

2.1.6 Fungsi Objektif

Fungsi objektif digunakan untuk menilai kualitas solusi yang dihasilkan. Fungsi ini dapat berupa nilai yang ingin dimaksimumkan, seperti keuntungan atau skor, maupun nilai yang ingin diminimumkan, seperti biaya atau jarak.

Agar greedy dapat menghasilkan solusi optimal, persoalan idealnya memiliki *greedy-choice property* dan *optimal substructure*. Namun, pada persoalan yang dinamis, greedy tidak selalu menjamin hasil optimal. Meskipun demikian, strategi ini tetap berguna ketika keputusan harus diambil secara cepat berdasarkan informasi yang tersedia saat itu [2, 3].

2.2 Cara Kerja Program Secara Umum

Program bot pada *Robocode Tank Royale* bekerja sebagai pengendali otomatis untuk sebuah tank virtual. Pemain tidak mengendalikan tank secara langsung selama pertandingan, melainkan menuliskan kode yang berperan sebagai “otak” bot. Kode inilah yang menentukan cara bot bergerak, memindai musuh, mengarahkan meriam, menembak, dan merespons kejadian selama pertempuran.

2.2.1 Siklus Pertandingan

Pertandingan dalam *Robocode Tank Royale* terdiri atas beberapa *round*. Setiap *round* dibagi lagi menjadi banyak *turn*, yaitu satuan waktu terkecil dalam permainan. Pada setiap *turn*, bot menerima informasi terbaru mengenai kondisi arena dan harus segera menentukan aksi yang akan dilakukan.

Informasi yang diterima bot dapat berupa:

- posisi dan arah badan bot;
- arah meriam dan arah radar;
- kecepatan gerak dan sisa energi bot;
- hasil pemindaian musuh, seperti jarak, arah relatif, dan energi lawan;
- kejadian khusus, misalnya terkena peluru, menabrak dinding, atau bertabrakan dengan bot lain.

2.2.2 Pemrosesan Informasi Arena

Sebelum mengambil keputusan, bot terlebih dahulu mengolah informasi dari arena. Radar menjadi bagian penting karena musuh hanya dapat diketahui jika berada dalam jangkauan dan sudut sapuan radar. Oleh karena itu, radar perlu diputar secara berkala agar bot tidak kehilangan informasi posisi lawan.

Setelah musuh terdeteksi, bot menyimpan atau langsung menggunakan informasi tersebut untuk menentukan prioritas aksi. Contohnya, jika musuh berada dalam jarak dekat, bot dapat memprioritaskan tembakan atau manuver menghindar. Jika bot berada dekat dinding, bot perlu mengubah arah gerak agar tidak menerima *wall damage*.

2.2.3 Pengambilan Keputusan Greedy

Algoritma greedy diterapkan dengan memilih aksi yang paling menguntungkan berdasarkan kondisi saat ini. Bot tidak mencoba semua kemungkinan aksi hingga akhir pertandingan, tetapi menilai beberapa kandidat aksi pada *turn* berjalan.

Kandidat aksi yang dapat dipilih antara lain:

- memutar radar untuk mencari atau mengunci posisi musuh;
- mengarahkan meriam ke posisi musuh terdekat atau paling mudah ditembak;
- menembak dengan daya tertentu sesuai jarak dan peluang mengenai sasaran;
- bergerak maju, mundur, atau berbelok untuk menjaga posisi aman;
- menjauh dari dinding atau menghindari tabrakan dengan bot lain.

Setiap kandidat diberi nilai berdasarkan heuristik yang digunakan. Aksi dengan nilai terbaik dipilih sebagai keputusan pada *turn* tersebut. Dengan cara ini, bot dapat mengambil keputusan secara cepat dan tetap sesuai dengan fungsi objektif permainan, yaitu mengumpulkan skor setinggi mungkin.

2.2.4 Eksekusi Aksi Bot

Setelah keputusan dipilih, bot mengirimkan perintah ke *game engine*. Perintah yang dikirim dapat berupa perintah gerak, putar badan, putar radar, putar meriam, atau tembak. Karena setiap *turn* memiliki batas waktu eksekusi, proses pengambilan keputusan harus sederhana dan efisien. Hal ini sesuai dengan karakteristik greedy yang ringan secara komputasi.

Dalam praktiknya, bot juga perlu mempertimbangkan energi. Menembak dapat menghasilkan skor jika peluru mengenai musuh, tetapi setiap tembakan juga mengurangi energi bot. Oleh sebab itu, bot tidak selalu menembak dengan daya besar. Daya tembak dapat disesuaikan dengan jarak musuh, arah gerak musuh, dan kondisi energi bot sendiri.

2.2.5 Menjalankan Program pada Game Engine

Program dijalankan melalui *engine Robocode Tank Royale* hasil modifikasi asisten. Langkah umum menjalankan bot adalah sebagai berikut.

1. Menjalankan GUI dengan berkas *robocode-tankroyale-gui-0.30.0.jar*.
2. Mengatur *Bot Root Directories* agar menunjuk ke direktori yang berisi folder bot.
3. Membuat *battle* baru melalui menu *Battle*.
4. Melakukan *boot* pada bot yang akan digunakan.
5. Menambahkan bot ke daftar peserta pertandingan.
6. Memulai pertandingan dengan menekan tombol *Start Battle*.

Setelah pertandingan dimulai, *engine* mengatur jalannya *turn*, mengirim informasi arena kepada bot, menerima perintah dari bot, menghitung perubahan energi, tabrakan, peluru, dan skor, lalu menentukan hasil akhir berdasarkan akumulasi skor selama pertempuran.

BAB 3

APLIKASI STRATEGI GREEDY

3.1 Pemetaan Persoalan ke Elemen Algoritma Greedy

Pada permainan *Robocode Tank Royale*, bot harus mengambil keputusan pada setiap *turn* berdasarkan informasi arena yang tersedia. Keputusan tersebut dapat dipetakan ke dalam elemen algoritma greedy sebagai berikut.

3.1.1 Himpunan Kandidat

Himpunan kandidat adalah seluruh aksi yang dapat dilakukan bot, seperti memutar radar, mengarahkan meriam, menembak dengan daya tertentu, bergerak maju atau mundur, berbelok, menghindari dinding, dan merespons tabrakan.

3.1.2 Himpunan Solusi

Himpunan solusi adalah aksi-aksi yang dipilih bot selama pertandingan. Solusi terbentuk sebagai rangkaian keputusan pada setiap *turn*, misalnya memindai lawan, memilih daya tembak, melakukan manuver menghindar, atau mendekati lawan ketika kondisi menguntungkan.

3.1.3 Fungsi Solusi

Fungsi solusi menentukan apakah keputusan pada suatu *turn* sudah menghasilkan aksi yang dapat dijalankan oleh bot. Dalam konteks permainan, solusi sementara dianggap terbentuk ketika bot telah memilih aksi terbaik untuk kondisi saat itu.

3.1.4 Fungsi Seleksi

Fungsi seleksi memilih aksi yang paling menguntungkan berdasarkan heuristik bot. Contohnya, bot dapat memilih tembakan berdaya besar saat jarak lawan dekat, memilih tembakan kecil saat lawan jauh, atau memilih manuver menghindar saat berada dekat dinding.

3.1.5 Fungsi Kelayakan

Fungsi kelayakan memastikan aksi yang dipilih masih aman dan mungkin dilakukan. Contohnya, bot tidak menembak dengan daya besar ketika energi rendah, tidak terus maju ketika berada dekat dinding, dan tidak melakukan *ramming* jika energi lawan masih tinggi.

3.1.6 Fungsi Objektif

Fungsi objektif pada tugas ini adalah memperoleh skor setinggi mungkin. Skor dapat ditingkatkan melalui tembakan yang mengenai lawan, bertahan hidup lebih lama, mengurangi kerusakan akibat dinding atau tabrakan, dan memanfaatkan peluang menyerang ketika kondisi menguntungkan.

3.2 Eksplorasi Empat Alternatif Solusi Greedy

Empat bot yang dibuat memiliki fokus greedy yang berbeda. Perbedaan ini sengaja dibuat agar setiap alternatif memiliki heuristik sendiri, sesuai ketentuan tugas besar bahwa strategi greedy pada tiap bot tidak boleh sama. Bot yang dibandingkan adalah *CodeCey*, *AdaptiveBot*, *Bot1*, dan *Bot2*. Dari keempat alternatif tersebut, *Bot1* dipilih sebagai bot utama karena hasil pengujian *1 vs 1* menunjukkan performa paling stabil dibandingkan bot alternatif lain.

3.2.1 Bot Alternatif 1: CodeCey

Greedy by Smart Gunner

CodeCey menggunakan strategi greedy yang mengutamakan akurasi tembakan. Bot memilih daya tembak berdasarkan jarak musuh, energi musuh, dan energi bot sendiri. Bot juga tidak langsung menembak setiap kali musuh terlihat, tetapi mengecek apakah arah meriam sudah cukup dekat dengan target. Dengan begitu, energi peluru tidak terlalu boros.

Pemetaan Elemen Greedy

1. **Himpunan kandidat:** memutar radar, mengarahkan meriam, memilih daya tembak, menembak, melakukan *dodge*, membalik arah, atau melakukan *ramming* ketika lawan sangat lemah.
2. **Himpunan solusi:** aksi terbaik pada kondisi saat itu, misalnya menembak menggunakan hasil fungsi ChooseFirePower atau menghindari ketika lawan terlalu dekat.
3. **Fungsi solusi:** keputusan dianggap terbentuk ketika bot menentukan akan menembak, menghindari, atau mengejar lawan pada *turn* tersebut.
4. **Fungsi seleksi:** memilih aksi berdasarkan jarak, selisih arah meriam, energi sendiri, dan energi musuh.
5. **Fungsi kelayakan:** tembakan hanya dilakukan jika energi cukup dan arah meriam cukup presisi. *Ramming* hanya dipilih jika energi bot jauh lebih unggul.
6. **Fungsi objektif:** memperbesar skor dari *bullet damage* sambil menjaga bot tetap hidup.

Analisis Efisiensi dan Efektivitas

Setiap pemindaian musuh hanya diproses dengan beberapa operasi hitung dan percabangan, sehingga kompleksitasnya adalah $O(1)$ per pemindaian. Strategi ini cukup hemat energi dan lebih hati-hati, tetapi pada pengujian strategi ini kurang agresif dibandingkan *Bot1*. Akibatnya, kesempatan menyerang dapat hilang ketika bot terlalu lama menunggu arah meriam benar-benar presisi.

3.2.2 Bot Alternatif 2: AdaptiveBot

Greedy by Adaptive Movement and Distance Fire

AdaptiveBot berfokus pada pergerakan adaptif dan penghindaran dinding. Ketika bot terlalu dekat dengan batas arena, bot langsung memilih aksi kembali ke tengah arena. Jika posisi masih aman, bot bergerak secara acak agar tidak mudah ditebak. Saat musuh terpindai, daya tembak dipilih berdasarkan jarak dan sisa energi.

Pemetaan Elemen Greedy

1. **Himpunan kandidat:** bergerak maju, bergerak mundur, berbelok acak, kembali ke tengah arena, membalik arah, dan menembak dengan daya 1, 2, atau 3.
2. **Himpunan solusi:** aksi gerak aman dan daya tembak yang dipilih pada kondisi saat itu.
3. **Fungsi solusi:** solusi terbentuk ketika bot sudah memilih gerakan dan serangan untuk satu kondisi arena.
4. **Fungsi seleksi:** prioritas pertama adalah menjauh dari dinding. Jika aman, bot bergerak acak dan menembak sesuai jarak musuh.
5. **Fungsi kelayakan:** gerakan dianggap layak jika menjauhkan bot dari dinding. Tembakan kuat hanya layak ketika musuh dekat dan energi bot masih cukup.
6. **Fungsi objektif:** mempertahankan hidup selama mungkin dan tetap menghasilkan skor melalui tembakan.

Analisis Efisiensi dan Efektivitas

Pengecekan posisi terhadap dinding dan pemilihan daya tembak dilakukan secara konstan, sehingga kompleksitasnya adalah $O(1)$ per iterasi. Strategi ini cukup baik untuk menghindari dinding, tetapi gerakan acak membuat posisi tembak tidak selalu optimal. Pada pengujian, *AdaptiveBot* masih dapat memberi perlawanan, tetapi total skornya belum mengungguli *Bot1*.

3.2.3 Bot Alternatif 3: Bot2

Greedy by Strafing and Dynamic Fire

Bot2 merupakan bot alternatif tambahan. Strategi utamanya adalah menembak dengan daya dinamis berdasarkan jarak, lalu melakukan gerakan *strafe* kiri-kanan agar tidak diam setelah menembak. Jika musuh dekat, bot menembak dengan daya besar dan bergerak menyamping. Jika musuh lebih jauh, bot tetap mendekat sambil menjaga pola gerak berbelok.

Pemetaan Elemen Greedy

1. **Himpunan kandidat:** memutar radar, mengarahkan meriam, menembak dengan daya 1, 2, atau 3, melakukan *strafe*, maju, mundur, dan berbelok saat menabrak.
2. **Himpunan solusi:** kombinasi tembakan dan gerak menyamping yang dipilih setelah musuh terpindai.
3. **Fungsi solusi:** solusi terbentuk ketika bot sudah menentukan daya tembak dan pola gerak berdasarkan jarak musuh.
4. **Fungsi seleksi:** memilih daya 3 untuk jarak dekat, daya 2 untuk jarak menengah, dan daya 1 untuk jarak jauh. Gerakan *strafe* dipilih agar bot tidak menjadi sasaran diam.
5. **Fungsi kelayakan:** aksi dianggap layak jika bot masih dapat bergerak dan tidak sedang terjebak di dinding. Saat menabrak dinding, bot mundur dan berbelok besar.
6. **Fungsi objektif:** menghasilkan kerusakan sebesar mungkin sambil mengurangi peluang terkena tembakan langsung.

Analisis Efisiensi dan Efektivitas

Strategi *Bot2* juga memiliki kompleksitas $O(1)$ per pemindaian karena hanya membandingkan jarak dengan beberapa batas nilai. Secara konsep, *strafe movement* dapat membantu bot menghindar. Namun, dari hasil pertandingan, pola gerak *Bot2* belum cukup efektif menghadapi *Bot1*. Bot sering kalah cepat dalam menghasilkan *bullet damage* dan tidak memperoleh nilai *survival*.

3.2.4 Bot Alternatif 4: Bot1

Greedy by Distance-Based Attack

Bot1 menggunakan strategi serangan berbasis jarak. Ketika musuh terpindai, bot langsung menentukan daya tembak dan gerak maju berdasarkan jarak musuh. Semakin dekat musuh, semakin besar daya tembak yang digunakan. Strategi ini sederhana, tetapi cepat dan agresif sehingga cocok untuk pertandingan *1 vs 1*.

Pemetaan Elemen Greedy

1. **Himpunan kandidat:** memutar badan untuk memindai, mengarahkan meriam, menembak dengan daya 1, 2, atau 3, maju mendekati lawan, mundur saat terlalu dekat, dan menghindari dinding.
2. **Himpunan solusi:** aksi serangan yang dipilih berdasarkan jarak musuh, yaitu tembakan besar untuk jarak dekat, tembakan sedang untuk jarak menengah, dan tembakan kecil untuk jarak jauh.
3. **Fungsi solusi:** solusi terbentuk ketika bot telah memilih kombinasi daya tembak dan arah gerak setelah musuh terpindai.
4. **Fungsi seleksi:** memilih daya tembak terbesar yang masih sesuai dengan jarak musuh. Jika musuh terlalu dekat, bot mundur dan berbelok untuk menjaga jarak.
5. **Fungsi kelayakan:** aksi dianggap layak jika bot dapat menembak musuh yang terpindai dan tetap merespons tabrakan atau dinding.
6. **Fungsi objektif:** memaksimalkan skor melalui serangan cepat dan keputusan sederhana berdasarkan jarak.

Analisis Efisiensi dan Efektivitas

Strategi ini hanya memeriksa jarak musuh dan memilih salah satu aksi tetap, sehingga kompleksitas setiap pemindaian adalah $O(1)$. Pada hasil pengujian, strategi ini menjadi paling efektif karena serangannya langsung dan tidak terlalu banyak menunggu kondisi lain. Kekurangannya, bot masih sederhana dan belum mempertimbangkan energi lawan secara detail.

3.3 Strategi Greedy yang Dipilih

Strategi greedy yang dipilih sebagai bot utama adalah strategi *distance-based attack* pada *Bot1*. Pemilihan ini didasarkan pada dua alasan. Pertama, aturan seleksinya sederhana sehingga bot cepat mengambil keputusan pada setiap *turn*. Kedua, hasil pengujian menunjukkan *Bot1* menang ketika diuji melawan tiga bot alternatif, yaitu *AdaptiveBot*, *Bot2*, dan *CodeCey*.

Pada strategi ini, jarak musuh menjadi heuristik utama. Jika musuh berada pada jarak dekat, bot memilih tembakan kuat karena peluang mengenai target lebih besar. Jika musuh berada pada jarak menengah, bot memilih tembakan sedang. Jika musuh jauh, bot menggunakan tembakan kecil sambil bergerak maju untuk memperpendek jarak. Keputusan ini memang belum sempurna, tetapi sudah sesuai dengan prinsip greedy karena selalu memilih aksi yang dianggap paling menguntungkan pada kondisi saat itu.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Empat Alternatif Solusi

Bagian ini menjelaskan implementasi empat bot dalam bentuk pseudocode. Pseudocode dibuat berdasarkan fungsi utama dan event penting pada masing-masing bot. Bot utama yang dipilih adalah *Bot1*, sedangkan *CodeCey*, *AdaptiveBot*, dan *Bot2* menjadi bot alternatif pembandingan.

4.1.1 Bot Alternatif 1: CodeCey

Fungsi Utama

```
1 function Run()
2     atur warna bot
3     movingForward <- true
4     putar radar terus-menerus
5
6     while bot masih berjalan do
7         bergerak sangat jauh sesuai moveDirection
8         putar badan ke kanan dan kiri secara bergantian
9         tunggu sampai gerakan selesai
10    end while
11 end function
```

Kode 4.1: Pseudocode fungsi utama CodeCey

Fungsi Saat Musuh Terpindai

```
1 function OnScannedBot(enemy)
2   if enemy adalah teman then return
3
4   distance <- jarak ke enemy
5   gunBearing <- arah meriam ke enemy
6   bodyBearing <- arah badan ke enemy
7   arahkan meriam ke enemy
8   firePower <- ChooseFirePower(distance , enemy.energy)
9
10  cek presisi arah meriam berdasarkan jarak
11  cek apakah energi bot cukup untuk menembak
12
13  if tembakan layak then
14    tembak dengan firePower
15  end if
16
17  if musuh terlalu dekat then
18    if musuh lemah dan energi bot unggul then maju untuk
19    ram
20    else lakukan QuickDodge(bodyBearing)
21  end if
22 end function
```

Kode 4.2: Pseudocode strategi greedy CodeCey saat musuh terpindai

Fungsi Pemilihan Daya Tembak

```
1 function ChooseFirePower(distance , enemyEnergy)
2   pilih power awal berdasarkan jarak
3   kecilkan power jika enemyEnergy rendah agar tidak overkill
4   kecilkan power jika energi bot rendah agar tetap bertahan
5   return power pada rentang 0.35 sampai 3.0
6 end function
```

Kode 4.3: Pseudocode pemilihan daya tembak CodeCey

4.1.2 Bot Alternatif 2: AdaptiveBot

Fungsi Utama dan Pergerakan Adaptif

```
1 function Run()
2     atur warna bot
3     movingForward <- true
4
5     while bot masih berjalan do
6         AdaptiveMovement()
7     end while
8 end function
9
10 function AdaptiveMovement()
11     margin <- 120
12
13     if posisi bot dekat dinding then
14         hentikan gerakan
15         mundur atau maju menjauh dari dinding
16         putar badan menuju tengah arena
17         maju ke arah tengah arena
18         return
19     end if
20
21     pilih jarak dan sudut gerak secara acak
22     sesekali balik arah gerak
23     jalankan gerakan sampai selesai atau dekat dinding
24 end function
```

Kode 4.4: Pseudocode pergerakan AdaptiveBot

Fungsi Saat Musuh Terpindai

```
1 function OnScannedBot(enemy)
2     distance <- jarak ke enemy
3
4     if distance > 250 or energy bot < 20 then tembak daya 1
5     else if distance > 100 then tembak daya 2
6     else tembak daya 3
7 end function
```

Kode 4.5: Pseudocode pemilihan daya tembak AdaptiveBot

Fungsi Respons Tabrakan

```
1 function OnHitWall(event)
2     ReverseDirection()
3 end function
4
5 function OnHitBot(event)
6     if bot ditabrak atau melakukan tabrakan then
7         ReverseDirection()
8     end if
9 end function
```

Kode 4.6: Pseudocode respons tabrakan AdaptiveBot

4.1.3 Bot Alternatif 3: Bot2

Fungsi Utama

```
1 function Run()
2     atur warna bot
3
4     while bot masih berjalan do
5         putar radar 360 derajat untuk mencari musuh
6     end while
7 end function
```

Kode 4.7: Pseudocode fungsi utama Bot2

Fungsi Saat Musuh Terpindai

```
1 function OnScannedBot(enemy)
2     distance <- jarak ke enemy
3     arahkan meriam ke enemy
4
5     if distance < 150 then tembak daya 3
6     else if distance < 300 then tembak daya 2
7     else tembak daya 1
8
9     if distance < 250 then
10         putar kanan 90 derajat
11         maju 80
12         putar kiri 90 derajat
```

```

13     else
14         putar kanan 30 derajat
15         maju 100
16         putar kiri 60 derajat
17     end if
18 end function

```

Kode 4.8: Pseudocode strategi greedy Bot2 saat musuh terpindai

Fungsi Respons Tabrakan

```

1 function OnHitBot(event)
2     tembak daya 3
3     mundur 60
4     putar kanan 45 derajat
5 end function
6
7 function OnHitWall(event)
8     mundur 120
9     putar kanan 120 derajat
10 end function

```

Kode 4.9: Pseudocode respons tabrakan Bot2

4.1.4 Bot Alternatif 4: Bot1

Fungsi Utama

```

1 function Run()
2     atur warna bot
3
4     while bot masih berjalan do
5         putar badan 360 derajat untuk mencari musuh
6     end while
7 end function

```

Kode 4.10: Pseudocode fungsi utama Bot1

Fungsi Saat Musuh Terpindai

```
1 function OnScannedBot(enemy)
2     distance <- jarak ke enemy
3
4     if distance < 150 then
5         tembak daya 3
6         maju 20
7     else if distance < 400 then
8         tembak daya 2
9         maju 40
10    else
11        tembak daya 1
12        maju 60
13    end if
14
15    if distance < 100 then
16        mundur 80
17        putar kanan 45 derajat
18    else
19        arahkan meriam ke enemy
20        maju 30
21    end if
22 end function
```

Kode 4.11: Pseudocode strategi greedy Bot1 saat musuh terpindai

Fungsi Respons Tabrakan

```
1 function OnHitBot(event)
2     tembak daya besar
3     mundur 50
4 end function
5
6 function OnHitWall(event)
7     mundur 100
8     putar kanan 90 derajat
9 end function
```

Kode 4.12: Pseudocode respons tabrakan Bot1

4.2 Struktur Data, Fungsi, dan Prosedur Bot

Struktur program setiap bot memanfaatkan event dari API *Robocode Tank Royale*.

Fungsi Run digunakan sebagai alur utama, sedangkan fungsi event seperti OnScannedBot, OnHitWall, OnHitBot, dan OnHitByBullet digunakan untuk merespons kondisi arena.

Data utama yang digunakan bot adalah posisi bot, posisi musuh, jarak musuh, arah meriam, arah badan, energi, dan status tabrakan.

Tabel 4.1: Ringkasan fungsi utama pada empat bot

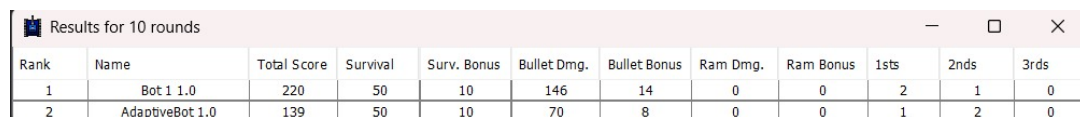
Bot	Fungsi/Prosedur	Kegunaan
CodeCey	Run	Mengatur radar berputar terus dan membuat gerakan zig-zag agar sulit ditembak.
CodeCey	ChooseFirePower	Memilih daya tembak berdasarkan jarak, energi musuh, dan energi bot sendiri.
AdaptiveBot	AdaptiveMovement	Menghindari dinding dan menghasilkan gerakan acak agar bot tidak mudah diprediksi.
AdaptiveBot	SmartFire	Memilih daya tembak berdasarkan jarak musuh dan energi bot.
Bot2	OnScannedBot	Mengarahkan meriam, memilih daya tembak dinamis, lalu melakukan gerak <i>strafe</i> .
Bot2	OnHitWall	Mundur dan berbelok besar agar bot keluar dari area dinding.
Bot1	OnScannedBot	Memilih daya tembak dan gerakan berdasarkan jarak musuh.
Bot1	OnHitWall	Mundur dan berbelok untuk mengurangi risiko terjebak di dinding.

4.3 Pengujian Bot

Pengujian dilakukan dalam mode *1 vs 1* selama 10 ronde. Bot utama yang diuji adalah *Bot1*, sedangkan bot pembandingnya adalah *AdaptiveBot*, *Bot2*, dan *CodeCey*. Tujuan pengujian ini adalah melihat apakah strategi greedy berbasis jarak pada *Bot1* lebih efektif dibandingkan strategi alternatif yang dibuat.

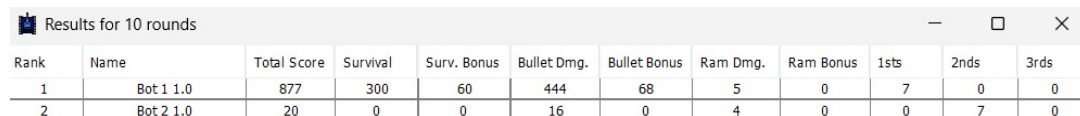
Tabel 4.2: Rangkuman hasil pengujian 1 vs 1 selama 10 ronde

No.	Pertandingan	Skor Bot1	Skor Lawan	Kesimpulan
1	Bot1 vs AdaptiveBot	220	139	Bot1 menang. AdaptiveBot masih bertahan cukup baik, tetapi skor akhir Bot1 tetap lebih tinggi.
2	Bot1 vs Bot2	877	20	Bot1 menang dominan; Bot2 tidak memperoleh nilai <i>survival</i> .
3	Bot1 vs CodeCey	573	36	Bot1 menang; CodeCey terlalu hati-hati sehingga skor akhirnya rendah.



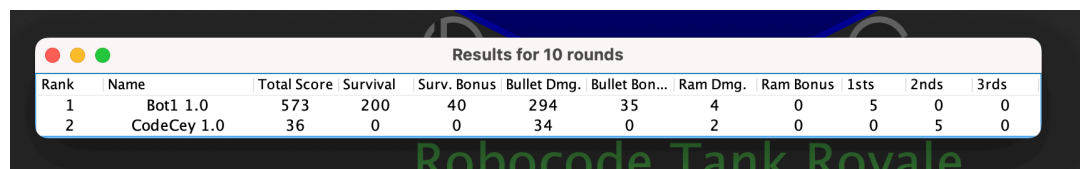
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Bot 1 1.0	220	50	10	146	14	0	0	2	1	0
2	AdaptiveBot 1.0	139	50	10	70	8	0	0	1	2	0

Gambar 4.1: Hasil pertandingan Bot1 melawan AdaptiveBot



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Bot 1 1.0	877	300	60	444	68	5	0	7	0	0
2	Bot 2 1.0	20	0	0	16	0	4	0	0	7	0

Gambar 4.2: Hasil pertandingan Bot1 melawan Bot2



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bon...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Bot 1 1.0	573	200	40	294	35	4	0	5	0	0
2	CodeCey 1.0	36	0	0	34	0	2	0	0	5	0

Gambar 4.3: Hasil pertandingan Bot1 melawan CodeCey

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian, *Bot1* menjadi bot terbaik dari empat alternatif yang dibuat. Pada pertandingan melawan *AdaptiveBot*, Bot1 memperoleh skor 220, sedangkan *AdaptiveBot* memperoleh skor 139. Selisihnya tidak terlalu jauh karena kedua bot sama-sama memperoleh nilai *survival* sebesar 50. Namun, Bot1 unggul karena menghasilkan *bullet damage* dan *bullet bonus* yang lebih besar.

Pada pertandingan melawan *Bot2*, Bot1 memperoleh skor 877, sedangkan *Bot2* hanya memperoleh skor 20. Hasil ini menunjukkan bahwa strategi *strafe* pada *Bot2* belum cukup kuat. Bot2 bergerak setelah menembak, tetapi pola gerakannya masih mudah

dimanfaatkan oleh Bot1 yang langsung menyerang berdasarkan jarak. Selain itu, Bot2 tidak mendapatkan nilai *survival*, sehingga total skornya sangat rendah.

Pada pertandingan melawan *CodeCey*, Bot1 memperoleh skor 573, sedangkan *CodeCey* memperoleh skor 36. *CodeCey* sebenarnya memiliki seleksi tembakan yang lebih rinci, tetapi strategi tersebut menjadi kurang agresif pada pertandingan *1 vs 1*. Bot1 yang lebih sederhana justru lebih cepat menembak dan mengumpulkan skor.

Secara umum, hasil pengujian menunjukkan bahwa strategi greedy yang terlalu banyak mempertimbangkan kondisi tidak selalu lebih baik. Dalam kasus ini, heuristik sederhana berbasis jarak lebih cocok karena keputusan dapat diambil cepat. Meski begitu, Bot1 masih memiliki kekurangan. Bot1 belum mempertimbangkan energi lawan, belum memprediksi arah gerak musuh, dan respons menghindarnya masih sederhana. Jika dikembangkan lagi, Bot1 dapat digabungkan dengan pemilihan daya tembak yang lebih hemat seperti *CodeCey* serta penghindaran dinding yang lebih aman seperti *AdaptiveBot*.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang dilakukan, dapat disimpulkan bahwa algoritma greedy dapat diterapkan pada bot *Robocode Tank Royale* dengan cara memilih aksi terbaik berdasarkan kondisi saat itu. Empat bot yang dibuat menggunakan heuristik berbeda, yaitu akurasi tembakan pada *CodeCey*, pergerakan adaptif pada *AdaptiveBot*, *strafe* dan tembakan dinamis pada *Bot2*, serta serangan berbasis jarak pada *Bot1*.

Dari hasil pengujian *1 vs 1*, *Bot1* menjadi bot utama yang paling efektif. *Bot1* menang melawan *AdaptiveBot*, *Bot2*, dan *CodeCey*. Keunggulan utama *Bot1* adalah keputusan yang cepat dan agresif, sehingga bot dapat segera menembak berdasarkan jarak musuh tanpa proses perhitungan yang rumit.

5.2 Saran

Untuk pengembangan berikutnya, strategi *Bot1* dapat diperbaiki dengan menambahkan prediksi arah gerak musuh, pengaturan daya tembak berdasarkan energi sendiri dan energi lawan, serta penghindaran dinding yang lebih halus. Selain itu, pengujian juga sebaiknya dilakukan melawan lebih banyak tipe bot dan pada jumlah ronde yang lebih besar agar hasil evaluasi lebih kuat.

BAB A

LAMPIRAN

A.1 Tautan Repository GitHub

Repository GitHub: https://github.com/chesyamd/Tubes_BukanOrangKampung.git

BIBLIOGRAPHY

- [1] MIT OpenCourseWare, “Introduction to Algorithms: Greedy Algorithms,” [Online]. Available: <https://ocw.mit.edu/courses/6-046j-introduction-to-algorithms-sma-5503-fall-2005/resources/lecture-videos/>, 2005, accessed: May 15, 2026.
- [2] Princeton University, “Greedy Algorithms,” [Online]. Available: https://competitive-programming.cs.princeton.edu/greedy_s24, 2024, accessed: May 15, 2026.
- [3] TU Delft OpenCourseWare, “Greedy Algorithms,” [Online]. Available: <https://ocw.tudelft.nl/course-lectures/3-greedy-algorithms/>, 2026, accessed: May 15, 2026.